

CardioSurv

Task Assignment Plan

Project: Cardiovascular Risk Classification, Treatment Recommendation, Survival Prediction
Course: AIT201 Applied Machine Learning, Xiamen University Malaysia, Session 202604
Team size: 6 members • **Deadline:** 25 June 2026

Format: 6 Sections × 6 Tasks = 36 tasks total. Each task is tagged with its owner and target week.

Contents

1	Section 1 — Data Pipeline & Database	2
2	Section 2 — ML Part 1: Risk Classification	4
3	Section 3 — ML Part 2, Clinical Formulas & Survival	6
4	Section 4 — Backend API & Deployment	8
5	Section 5 — Frontend Interface	10
6	Section 6 — Evaluation, Report & Presentation	12
7	Summary Tables	14

Badge Legend

Mn solo One member owns this task independently. **Mn + Mm pair** Two members work together (integration point). **All members** Every member contributes.

Week 1 **Week 2** **Week 3** **Week 4** **Week 5** **Week 6**

Target week for each task.

1. Section 1 — Data Pipeline & Database

Section 1 — Data Pipeline & Database

Primary owner: M1 • 6 tasks

T1.1 Repository setup & raw data download

M1 solo Week 1

WHAT TO BUILD

Initialise the GitHub repo skeleton with all folders, `.gitignore`, `requirements.txt`, and `README`. Then write `src/data/download.py` to pull the Kaggle Heart Failure Prediction dataset and the UCI Cleveland Heart Disease dataset into `data/raw/`. This is the very first task — the whole team depends on the repo being live before anyone else can push code.

DELIVERABLES

GitHub repo live on main data/raw/ with 2 CSVs download.py script

T1.2 Schema documentation & dataset merging

M1 solo Week 1

WHAT TO BUILD

Compare both datasets side-by-side. Document every column mapping, type mismatch, and encoding difference in `docs/schemas.md`. Then write `src/data/merge.py` that renames columns, harmonises categoricals, and outputs `data/processed/merged.csv`. The schema document must be signed off by the whole team on Day 1 — it is the contract that prevents M2, M3, and M4 from building against incompatible column names.

DELIVERABLES

docs/schemas.md locked data/processed/merged.csv merge.py

T1.3 Data cleaning & EDA notebook

M1 solo Week 1

WHAT TO BUILD

Write `src/data/clean.py` to handle missing values, outliers, and type coercion. Build `notebooks/01_eda.ipynb` showing distributions, correlations, and class balance charts. The EDA notebook is the shared reference for the whole team to understand the data before building models.

DELIVERABLES

clean.py 01_eda.ipynb with figures

T1.4 Feature engineering & unit tests

M1 solo Week 2

WHAT TO BUILD

Implement `src/features/engineering.py` with three derived features: `age.bin()`, `bp_risk.level()`, and `hr_stress_index()`. Write `tests/test_features.py` covering edge cases for each function. Output `data/processed/features.csv` — this is the file M2 trains on, so it must be ready by the start of Week 2.

DELIVERABLES

engineering.py tests/test_features.py passing data/processed/features.csv

T1.5 Database models & migrations

M1 + M4 pair

Week 3

WHAT TO BUILD

Implement the four SQLAlchemy models (patients, predictions, recommendations, audit_logs) in `src/db/models.py`. Set up Alembic and generate the first migration file. Write `src/db/session.py` that reads `DATABASE_URL` from the environment. Document local Postgres setup via Docker Compose in the README. M1 owns the model definitions; M4 integrates them into the API.

DELIVERABLES

`src/db/models.py` `alembic/versions/` first migration `session.py`
`docker-compose.yml` working

T1.6 Seed data & integration pairing

M1 + M4 pair

Week 4

WHAT TO BUILD

Write `src/db/seed.py` to load processed patient rows into the database for testing. Pair with M4 to wire the DB session into the live API endpoints and verify that a `POST /api/predict` correctly inserts a patient row and a prediction row, and that `GET /api/history` returns those records.

DELIVERABLES

`seed.py` DB reads/writes confirmed end-to-end in API

2. Section 2 — ML Part 1: Risk Classification

Section 2 — ML Part 1: Risk Classification

Primary owner: M2 • 6 tasks

T2.1 Output schema agreement & classifier scaffold

M2 solo Week 1

WHAT TO BUILD

Get team sign-off (in the Day-1 meeting) on the exact prediction output format: `{patient_id, risk_category, confidence, model_version}`. Lock it in `docs/schemas.md`. Then write `src/models/part1_classifier.py` with stubs for `train()`, `predict()`, `evaluate()`, `save()`, and `load()` so M4 can import and call the module immediately without waiting for training to finish.

DELIVERABLES

Output schema locked in `docs/schemas.md` `part1_classifier.py` scaffold importable

T2.2 Mock dataset & preprocessing pipeline

M2 solo Week 1

WHAT TO BUILD

Generate a synthetic features CSV that matches the agreed schema so model development is not blocked while M1 finishes cleaning the real data. Build the full `sklearn` Pipeline: `StandardScaler` for numeric columns, `OneHotEncoder` for categoricals, 80/20 stratified train-test split. The pipeline object will be reused unchanged when real data arrives.

DELIVERABLES

Mock features CSV Preprocessing Pipeline object (`sklearn`)

T2.3 Random Forest baseline & XGBoost training

M2 solo Week 2

WHAT TO BUILD

Train a Random Forest classifier on `features.csv` and log accuracy, F1 (macro), AUC-ROC, and a confusion matrix. Then train an XGBoost classifier with default parameters on the same data. Compare both models on the same metrics table and pick the winner. Document the choice with rationale in `notebooks/02_part1_modeling.ipynb`.

DELIVERABLES

Trained RF model Trained XGBoost model Comparison table in notebook

T2.4 Hyperparameter tuning & cross-validation

M2 solo Week 3

WHAT TO BUILD

Run `GridSearchCV` or `Optuna` on the winning model. Perform 5-fold cross-validation and report mean $\pm$ std for every metric. Implement multi-class `predict_proba` so the API can return a confidence score (not just a label). Serialise the final model with `joblib.dump()` to `models/part1_classifier_v1.0.pkl`.

DELIVERABLES

`models/part1_classifier_v1.0.pkl` CV results table (mean $\pm$ std)

T2.5 API integration function & endpoint pairing

M2 + M4 pair

Week 4

WHAT TO BUILD

Wrap the trained model in a clean function `part1_predict(features_dict) → dict` that M4 can call with one line inside the FastAPI route. Pair with M4 to plug it into `POST /api/predict` and run end-to-end: send a JSON body, get a real `risk_category` and `confidence` back. This is Integration Milestone M2.

DELIVERABLES

`part1_predict()` importable function`/api/predict` returning real predictions**T2.6 Final accuracy push & evaluation figures**

M2 solo

Week 5

WHAT TO BUILD

Apply feature selection and class-weight adjustments if the model is below the target threshold. Generate the final confusion matrix heatmap and ROC curve PNGs at print resolution to `reports/figures/`. Write the Part 1 Methods and Results subsections of the final report. Prepare 3–4 slides on model performance for the presentation.

DELIVERABLES

`confusion_matrix.png``roc_curve.png`

Report Part 1 section

Slide content

3. Section 3 — ML Part 2, Clinical Formulas & Survival

Section 3 — ML Part 2, Clinical Formulas & Survival Primary owner: M3 • 6 tasks

T3.1 PhysioNet credentialing (Day 0 action) & BED formula

M3 solo Week 1

WHAT TO BUILD

Start CITI training on Day 0 before the first meeting at <https://about.citiprogram.org> — this takes 5–7 days and cannot be skipped or delayed. In parallel, implement `src/clinical/bed.py` with the BED formula:

$$\text{BED} = D_{\text{total}} \times \left(1 + \frac{d}{\alpha/\beta}\right)$$

where d is dose per fraction and $\alpha/\beta = 10$ Gy (default). Include input validation for dose ranges and fraction count consistency.

DELIVERABLES

CITI training enrolled (confirmed Day-1) bed.py with validation

T3.2 GRACE score, routing logic & unit tests

M3 solo Week 1

WHAT TO BUILD

Implement `src/clinical/grace.py` using the JAMA reference lookup tables for the GRACE Risk Score. Implement `src/clinical/routing.py`: if `risk_category == "High"` and `has_arrhythmia` → SBRT branch, else → Medication branch. Write `tests/test_bed.py` and `tests/test_grace.py` with known clinical reference values from the literature.

DELIVERABLES

grace.py routing.py test_bed.py passing test_grace.py passing

T3.3 Mock recommender for early API integration

M3 solo Week 2

WHAT TO BUILD

While waiting for MIMIC-IV access to be granted, build a mock version of `part2_recommend()` that accepts the real input schema and returns hardcoded but realistic output (`branch`, `intervention_type`, `survival_without`, `survival_with`). This lets M4 wire up `/api/recommend` immediately without being blocked. Mark outputs as mock clearly in the response payload.

DELIVERABLES

Mock `part2_recommend()` returning canned JSON M4 can integrate `/api/recommend`

T3.4 MIMIC-IV exploration & Part 2 feature set

M3 solo Week 3

WHAT TO BUILD

Once MIMIC-IV access is granted, explore the key tables: `admissions`, `prescriptions`, `diagnoses_icd`, `patients`, `chartevents`. Build the Part 2 feature set by merging Part 1 outputs (`risk_category`, `confidence`) with MIMIC-IV metadata (`comorbidities`, `prior_mi`, `arrhythmia_flag`). Output a clean, joined CSV for model training.

DELIVERABLES

`MIMIC-IV access confirmed` `Part 2 features CSV`**T3.5** XGBoost recommender & survival model`M3 solo` `Week 4`

WHAT TO BUILD

Train an XGBoost classifier to predict `intervention_type` and `intensity_level` from the Part 2 features. Build the survival model with `lifelines`: Kaplan–Meier curves stratified by treatment branch, Cox Proportional Hazards model outputting `survival_without` and `survival_with` (2-year probabilities). Implement counterfactual logic: predict survival under both branches and return both values in the API response.

DELIVERABLES

`models/part2_recommender_v1.0.pkl` `models/survival_cox_v1.0.pkl`**T3.6** API wiring & evaluation figures`M3 + M4 pair` `Week 5`

WHAT TO BUILD

Pair with M4 to replace the mock recommender with the real model inside `/api/recommend`. This is Integration Milestone M3. Generate KM survival curves and feature importance plots to `reports/figures/`. Write the Part 2, BED, GRACE, and Survival sections of the final report. Prepare slide content for the presentation.

DELIVERABLES

`/api/recommend returning real outputs` `km_curves.png` `Report Part 2 section`

4. Section 4 — Backend API & Deployment

Section 4 — Backend API & Deployment

Primary owner: M4 • 6 tasks

T4.1 FastAPI scaffold & API contract

M4 solo Week 1

WHAT TO BUILD

Initialise FastAPI in `src/api/main.py` with CORS configured for the frontend domain. Write `docs/api_contract.md` defining every endpoint, full request body shape, and response JSON with example values. Get team sign-off on Day 1 — this document is the contract that M5 builds against. Implement Pydantic request and response models in `src/api/schemas.py`.

DELIVERABLES

`src/api/main.py` running locally `docs/api_contract.md` team sign-off

T4.2 Mocked API live on Render

M4 solo Week 1

WHAT TO BUILD

Implement all five endpoints returning hardcoded JSON (no models, no DB yet). Deploy this mock to Render.com as a Web Service so M5 can start building the frontend against a real HTTPS URL from Day 1. Document the live URL in `docs/api_contract.md`. This enables the entire team to move in parallel without waiting for any model or database.

DELIVERABLES

Mock API live at `cardiosurv-api.onrender.com` All 5 endpoints returning canned JSON

T4.3 Real endpoints with PostgreSQL

M4 solo Week 2

WHAT TO BUILD

Wire SQLAlchemy session management using M1's `session.py`. Implement real `/api/predict`: validate input with Pydantic, insert a patient row, call M2's `part1.predict()`, insert a prediction row, return all IDs. Implement `/api/recommend` using M3's mock, `/api/history` with pagination, and `/api/patients/{id}` with full history. Verify all with Postman or curl.

DELIVERABLES

All endpoints hitting DB + models CRUD confirmed with Postman/curl

T4.4 Production deployment on Render

M4 solo Week 3

WHAT TO BUILD

Create a Render Web Service from the GitHub repo (Python 3.11, build command `pip install -r requirements.txt`, start command `uvicorn src.api.main:app`). Provision Render PostgreSQL free tier. Set `DATABASE_URL`, `MODEL_DIR`, and `CORS_ORIGINS` env vars. Enable auto-deploy on push to `main`. Run Alembic migrations on the production DB via the Render shell. Smoke-test the live URL with curl. This is Integration Milestone M4.

DELIVERABLES

Production API URL live Auto-deploy from GitHub working Migrations applied on prod DB

T4.5 Error handling, logging & rate limiting

M4 solo Week 4

WHAT TO BUILD

Add proper HTTP status codes and structured validation error responses (422 body shows which field failed and why). Add structured logging with `structlog` or `stdlib`. Implement rate limiting on `/api/predict` using `slowapi`. Performance-test with `locust` or `ab`. Cache model loading at application startup so individual predictions are fast rather than loading the `.pkl` file on every request.

DELIVERABLES

Structured error responses (422) Rate limiting on `/api/predict`
Model loaded once at startup

T4.6 Deployment runbook & production monitoring

M4 solo Week 5

WHAT TO BUILD

Write `docs/deployment.md`: how to deploy from scratch, how to run migrations, how to roll back a bad migration, and how to check logs on Render. During demo week, keep the production URL warm by pinging `/api/health` every 10 minutes before the presentation — the Render free tier sleeps after 15 minutes of idle and would cause a 30-second cold start during the live demo.

DELIVERABLES

`docs/deployment.md` complete Production URL warm during demo

5. Section 5 — Frontend Interface

Section 5 — Frontend Interface

Primary owner: M5 • 6 tasks

T5.1 Wireframes & HTML skeleton

M5 solo Week 1

WHAT TO BUILD

Sketch wireframes for the four pages: home, predict, results, history. Create the `frontend/` folder structure. Build `index.html` (landing page) and `predict.html` with all input fields matching the API schema: age, sex, resting BP (systolic/diastolic), cholesterol, fasting blood sugar, resting ECG, max heart rate, exercise angina, oldpeak, ST slope. Set up `css/styles.css` with a CSS variable system for consistent spacing and colours.

DELIVERABLES

Wireframes (paper or Figma) `index.html` `predict.html` `styles.css`

T5.2 Client-side validation & API JS layer

M5 solo Week 1

WHAT TO BUILD

Add client-side validation for every field in `predict.html` (value ranges, required fields, correct types) with clear inline error messages before submission. Write `frontend/js/api.js` with `fetch()` wrappers for all five API endpoints. The base URL must be a single configurable constant so switching from the mock API to the production URL is a one-line change.

DELIVERABLES

All fields validated before submit `api.js` with all 5 endpoint wrappers

T5.3 Results page & survival chart

M5 solo Week 2

WHAT TO BUILD

Build `results.html` showing: risk category badge (shape-coded as well as colour-coded for accessibility — e.g. circle, triangle, square for Low/Medium/High), a confidence progress bar, a treatment recommendation card, and a counterfactual survival comparison panel. Import Chart.js from CDN and build a bar chart comparing `survival_without` vs `survival_with`. Wire the predict form to submit to the mock API and navigate to results passing the response via `sessionStorage`.

DELIVERABLES

`results.html` with badge, bar, survival chart Form submits to mock API and shows results

T5.4 History page, modals & cross-browser testing

M5 solo Week 3

WHAT TO BUILD

Build `history.html` with a paginated table showing date, patient ID, risk category, recommendation, and survival improvement per record. Add a patient details modal triggered by clicking any row — the modal fetches `/api/patients/{id}` and shows the full record. Cross-browser test in Chrome, Firefox, and Safari. Fix layout regressions.

DELIVERABLES

`history.html` with pagination Patient details modal working

Passes in Chrome, Firefox, Safari

T5.5 Mobile responsive & accessibility audit

M5 solo Week 4

WHAT TO BUILD

Add media queries for 768px and 480px breakpoints so the full app is usable on a phone. Conduct an accessibility audit: semantic HTML tags (`<main>`, `<nav>`, `<section>`), alt text on every image, explicit `<label>` associations on every form input, keyboard navigability for all interactive elements, and ARIA labels on dynamic regions. Switch `API_BASE_URL` in `api.js` from the mock to the real production Render URL.

DELIVERABLES

UI usable on 375px mobile All inputs labelled Keyboard navigable
Wired to real production API

T5.6 Error states, deployment & final polish

M5 + M4 pair Week 5

WHAT TO BUILD

Add graceful error states for API failures (503 banner), server-side validation errors (field-level messages), and empty history (friendly empty state illustration). Deploy to Render Static Site or GitHub Pages. Coordinate CORS origins with M4 so the deployed frontend is whitelisted. Add loading skeletons while API calls are in flight and smooth page transitions. Take final screenshots for the report and slides.

DELIVERABLES

Frontend live at production URL Error states handled Screenshots for report and slides

6. Section 6 — Evaluation, Report & Presentation

Section 6 — Evaluation, Report & Presentation

Primary owner: M6 • 6 tasks

T6.1 Evaluation scaffolding & plot system

M6 solo Week 1

WHAT TO BUILD

Set up `src/evaluation/metrics.py` with two functions: `classification_metrics()` returning accuracy, F1 (macro), AUC-ROC; and `survival_metrics()` returning C-index and log-rank p -value. Set up `src/evaluation/plots.py` with functions for confusion matrix heatmap, ROC curve, Kaplan–Meier curves with 95% confidence bands, and feature importance bar charts. Run all plot functions on mock data to verify rendering. Establish a consistent colour palette (hex values) used for all figures in the report.

DELIVERABLES

`metrics.py` `plots.py` Sample figures in `reports/figures/`

T6.2 Report skeleton & introduction

M6 solo Week 2

WHAT TO BUILD

Start the report document (`.tex` or `.docx`) with a title page, locked section outline, and placeholder headings. Get team sign-off on the structure. During Week 2 when M1–M5 are at peak load, make yourself available to help whoever needs it (M5 frontend, M1 EDA). Begin drafting the Introduction and Problem Statement using the proposal as a base.

DELIVERABLES

Report skeleton with locked outline Introduction section draft

T6.3 Part 1 evaluation figures & datasets section

M6 + M2 pair Week 3

WHAT TO BUILD

Pull M2's trained model from `models/part1_classifier.v1.0.pkl` and run a complete evaluation: confusion matrix, ROC curve, cross-validation summary table. Generate all Part 1 figures to `reports/figures/` using the established colour palette. Write the Datasets section of the report by consolidating M1's EDA notebook notes and the schema documentation.

DELIVERABLES

All Part 1 figures (print-res PNG) Datasets section written

T6.4 Part 2 & survival figures + methods section

M6 + M3 pair Week 4

WHAT TO BUILD

Pull M3's models and generate: Kaplan–Meier curves stratified by treatment branch, Cox model coefficient forest plot, Part 2 feature importance bars. Write the Methods section of the report by consolidating inputs from M1 (data pipeline), M2 (Part 1 model), and M3 (Part 2, clinical formulas, survival modelling). Every pipeline step must be described precisely enough for reproducibility.

DELIVERABLES

All Part 2 + survival figures (print-res PNG) Methods section written

T6.5 Results, discussion & full report draft**M6 solo** **Week 5**

WHAT TO BUILD

Write the Results section referencing all generated figures with precise metric values. Write Discussion (what the results mean for cardiovascular risk prediction), Limitations (MIMIC-IV data constraints, model assumptions, label noise), Future Work, and Conclusion. Compile the full reference list in the required citation format. Send the complete draft PDF to the team for feedback by end of Week 5, with a 48-hour review deadline.

DELIVERABLES

`Full report draft PDF sent to team` `All sections complete including references`**T6.6 Final report, slide deck & demo video****All members** **Week 6**

WHAT TO BUILD

Incorporate all team feedback and export the final report PDF. Build the slide deck: title, problem, approach, datasets, Part 1 results, Part 2 results, live demo, limitations, Q&A. Record a demo video walkthrough (OBS or Loom) showing the full user journey: submit a patient in the UI, see the classification, see the recommendation, browse the history, open the auto-generated API docs at `/docs`. Run a timed rehearsal with the whole team and tighten any weak spots.

DELIVERABLES

`reports/final_report.pdf` `reports/slides.pptx` `Demo video link in README`

7. Summary Tables

All 36 Tasks at a Glance

Task	Name	Owner	Wk	Key Deliverable
T1.1	Repo setup & raw data download	M1 solo	1	GitHub repo + 2 raw CSVs
T1.2	Schema docs & dataset merging	M1 solo	1	schemas.md + merged.csv
T1.3	Data cleaning & EDA notebook	M1 solo	1	clean.py + 01_eda.ipynb
T1.4	Feature engineering & tests	M1 solo	2	features.csv + test_features.py
T1.5	DB models & migrations	M1 + M4 pair	3	models.py + first migration
T1.6	Seed data & integration	M1 + M4 pair	4	seed.py + DB end-to-end
T2.1	Output schema + classifier scaffold	M2 solo	1	Schema locked + scaffold
T2.2	Mock dataset & preprocessing	M2 solo	1	Mock CSV + sklearn Pipeline
T2.3	RF baseline & XGBoost training	M2 solo	2	Trained RF + XGBoost
T2.4	Hyperparameter tuning & CV	M2 solo	3	part1_classifier_v1.0.pkl
T2.5	API integration function	M2 + M4 pair	4	part1_predict() + /api/predict live
T2.6	Final accuracy + eval figures	M2 solo	5	ROC PNG + report section
T3.1	CITI training + BED formula	M3 solo	1	CITI enrolled + bed.py
T3.2	GRACE score + routing + tests	M3 solo	1	grace.py + routing.py + tests
T3.3	Mock recommender	M3 solo	2	Canned part2_recommend()
T3.4	MIMIC-IV exploration + features	M3 solo	3	MIMIC access + Part 2 CSV
T3.5	XGBoost recommender + survival	M3 solo	4	Two serialised .pkl models
T3.6	API wiring + eval figures	M3 + M4 pair	5	Real /api/recommend + KM curves
T4.1	FastAPI scaffold + API contract	M4 solo	1	main.py + api_contract.md
T4.2	Mock API live on Render	M4 solo	1	Live URL with canned JSON
T4.3	Real endpoints + PostgreSQL	M4 solo	2	All endpoints hitting DB
T4.4	Production deployment	M4 solo	3	Prod URL + auto-deploy
T4.5	Error handling + rate limiting	M4 solo	4	422 errors + slowapi
T4.6	Deployment runbook + monitoring	M4 solo	5	deployment.md
T5.1	Wireframes & HTML skeleton	M5 solo	1	4 wireframes + predict.html
T5.2	Validation & API JS layer	M5 solo	1	Validation + api.js
T5.3	Results page & survival chart	M5 solo	2	results.html + Chart.js bar
T5.4	History page + modals	M5 solo	3	history.html + modal

Task	Name	Owner	Wk	Key Deliverable
T5.5	Mobile responsive + ally audit	M5 solo	4	Mobile + labels + real API
T5.6	Error states + deployment	M5 + M4 pair	5	Frontend live + error states
T6.1	Eval scaffolding + plot system	M6 solo	1	metrics.py + plots.py
T6.2	Report skeleton + introduction	M6 solo	2	Outline + intro draft
T6.3	Part 1 figures + datasets section	M6 + M2 pair	3	Part 1 PNGs + datasets text
T6.4	Part 2 figures + methods section	M6 + M3 pair	4	Part 2 PNGs + methods text
T6.5	Results, discussion + draft	M6 solo	5	Full draft PDF to team
T6.6	Final report + slides + demo	All members	6	PDF + PPTX + video link

Integration Milestones

ID	Milestone	Target	Owners	Done means
M1	Mock end-to-end	End Week 1	All	Frontend hits mock API, gets canned JSON
M2	Real Part 1 wired	End Week 3	M1, M2, M4	Submit vitals → real risk category
M3	Real Part 2 wired	End Week 4	M3, M4	Risk category → recommendation + survival
M4	Production deploy	Mid Week 5	M4, M5	Live URL accessible from anywhere
M5	Demo-ready	End Week 5	All	Full flow works, 4 example cases pass
M6	Submission ready	24 June 2026	M6 + all	Report, slides, demo, repo submitted

End of Task Plan. 36 tasks across 6 sections. Questions: bring to weekly sync.